Disaster Relief Resource Management System

Course: Advanced Database Management System Practical

Database: MySQL

Project Description

The Disaster Relief Resource Management System (DRRMS) will be developed as a database project for storing and managing disaster events, shelters, donors, donated resources, relief requests, volunteers and resource distribution. Self-created dataset will be used to complete database design, populated and tested using the concepts covered in the ADBMS practical.

Objectives

1. To enable the design of a structured dataset using proper data modelling techniques.

2. To develop practical skills in SQL for database creation, manipulation and querying.

3. To enhance analytical thinking through multi-phase database development.

Phase 1 - Data Modelling and Dataset Design

1. Dataset: A small DRRMS dataset containing disaster events, shelters, donors, resources, requests, volunteers and distribution records. The same data will be used in all later phases.

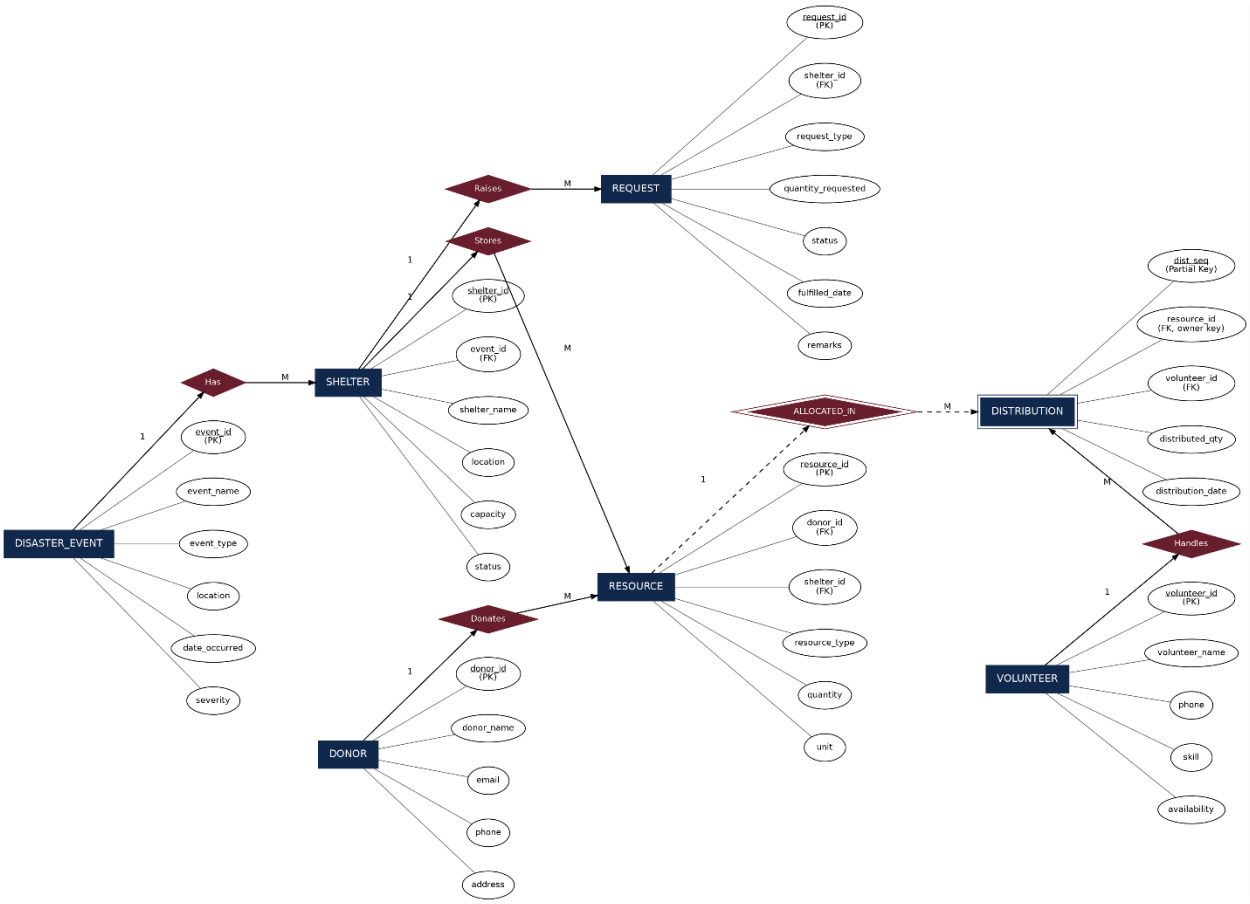
2. Entity and attribute identification: DISASTER_EVENT, SHELTER, DONOR, RESOURCE, REQUEST, VOLUNTEER and DISTRIBUTION as the main entities, with attributes required to store their respective details.

3. ER modelling: DISASTER_EVENT-SHELTER, DONOR-RESOURCE, SHELTER-RESOURCE, SHELTER-REQUEST, RESOURCE-DISTRIBUTION and VOLUNTEER-DISTRIBUTION will be modelled mainly as 1:M relationships.

4. Relational schema: Convert the ER model into tables. Primary keys will identify records and foreign keys will connect related tables.

5. Normalization: donor, shelter, request and distribution details will be stored separately so that repeated data is reduced and the schema remains in 3NF.

ER Diagram



Tables and Data Types

Table	Fields with Data Type	Relationship
DISASTER_EVENT	event_id INT PK AUTO_INCREMENT event_type VARCHAR(40) location VARCHAR(60) start_date DATE severity VARCHAR(15)	DISASTER_EVENT 1:M SHELTER
DONOR	donor_id INT PK AUTO_INCREMENT donor_name VARCHAR(60) NOT NULL phone VARCHAR(15) email VARCHAR(80) UNIQUE donor_type VARCHAR(20)	DONOR 1:M RESOURCE
VOLUNTEER	volunteer_id INT PK AUTO_INCREMENT volunteer_name VARCHAR(60) NOT NULL phone VARCHAR(15) skill VARCHAR(40)	VOLUNTEER 1:M DISTRIBUTION

Table	Fields with Data Type	Relationship
	availability_status VARCHAR(20)	
SHELTER	shelter_id INT PK AUTO_INCREMENT event_id INT FK location VARCHAR(60) capacity INT current_occupancy INT	FK event_id -> DISASTER_EVENT SHELTER 1:M RESOURCE SHELTER 1:M REQUEST
RESOURCE	resource_id BIGINT PK AUTO_INCREMENT donor_id INT FK shelter_id INT FK resource_type VARCHAR(30) quantity DECIMAL(10,2) unit VARCHAR(15)	FK donor_id -> DONOR FK shelter_id -> SHELTER RESOURCE 1:M DISTRIBUTION
REQUEST	request_id INT PK AUTO_INCREMENT shelter_id INT FK resource_type VARCHAR(30) quantity_requested DECIMAL(10,2) request_status VARCHAR(20) request_date DATE fulfilled_date DATE NULL	FK shelter_id -> SHELTER
DISTRIBUTION	distribution_id BIGINT PK AUTO_INCREMENT resource_id BIGINT FK volunteer_id INT FK distributed_qty DECIMAL(10,2) distributed_at DATETIME	FK resource_id -> RESOURCE FK volunteer_id -> VOLUNTEER

Relational Schema: DISASTER_EVENT(event_id) -> SHELTER(event_id); DONOR(donor_id) -> RESOURCE(donor_id); SHELTER(shelter_id) -> RESOURCE(shelter_id), REQUEST(shelter_id); RESOURCE(resource_id) -> DISTRIBUTION(resource_id); VOLUNTEER(volunteer_id) -> DISTRIBUTION(volunteer_id).

Phase 2 - Database Creation and Data Preparation

1. DDL: Create DRRMS_DB and all seven tables using CREATE DATABASE and CREATE TABLE. PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK constraints will be defined with the tables.

2. DML: Enter sample data using INSERT. UPDATE will be used when request status or resource quantity changes, while DELETE will be tested only where foreign-key relations are not affected.

3. Data cleaning and constraints: duplicate donor emails and shelter locations will be prevented, required IDs will not be NULL, and invalid negative quantity or capacity values will be restricted.

4. NULL handling: fulfilled_date will stay NULL while a request is pending and remarks can stay NULL when there is no comment. IS NULL / IS NOT NULL will be used to find such records.

5. Basic queries: SELECT, WHERE and ORDER BY will be used to display active shelters, pending requests, resources by type, volunteers by skill and distributions by date.

6. Aggregate functions: COUNT() will calculate requests per shelter, AVG() will calculate average resource quantity per type and SUM() will calculate total resources distributed.

Phase 3 - Data Querying and Analytical Operations

1. Joins: join DONOR + RESOURCE for donation details, SHELTER + REQUEST + DISASTER_EVENT for relief reports, and RESOURCE + DISTRIBUTION for allocation history.

2. Nested queries and subqueries: find shelters whose request count is above the average request count and resources whose distributed quantity is above the overall average.

3. View: create Shelter_Resource_Summary to display shelter name, disaster event, resource type, quantity and distributed quantity through one reusable query.

4. Query execution and optimization: use EXPLAIN on the shelter-request-event join. Indexes will be created on frequently searched columns such as RESOURCE.shelter_id, REQUEST.shelter_id and DISTRIBUTION.resource_id and the query plan will be checked again.

5. Function: create GetShelterRequestCount(shelter_id). It will receive a shelter ID, count the matching records in REQUEST and return the total requests raised by that shelter.

6. Procedure: create AddDistribution(resource_id, volunteer_id, distributed_qty). It will accept the distribution details, insert a new DISTRIBUTION record and reduce the available quantity in RESOURCE.

Functional Modules

Function / Task	Tables Used	How I will do it
Register donor and resource	DONOR, RESOURCE	I will add the donor first and then store the donated resource using donor_id as the foreign key.
Add and view shelters	DISASTER_EVENT, SHELTER	I will store shelter location, capacity and current occupancy against the linked disaster event.
Raise a request	SHELTER, REQUEST	I will record the resource type and quantity requested by a shelter and update its status as it is fulfilled.

Function / Task	Tables Used	How I will do it
Allocate distribution	RESOURCE, VOLUNTEER, DISTRIBUTION	I will select an available resource and volunteer, create the distribution record and reduce the resource quantity in one transaction.
Track disaster events	DISASTER_EVENT, SHELTER	I will store event type, location, start date and severity and link shelters opened for that event.
Generate summary reports	DONOR, RESOURCE, SHELTER, REQUEST, DISTRIBUTION	I will use joins, aggregates and the Shelter_Resource_Summary view to prepare donation, request and distribution summaries.

Phase 4 - Data Integrity and Transaction Handling

1. Transaction handling: when I allocate a resource to a distribution, I will start one transaction, insert the DISTRIBUTION record and reduce RESOURCE.quantity. A SAVEPOINT will be used before the final update. COMMIT will be used if both operations succeed and ROLLBACK if an error occurs.

2. Atomicity: the distribution insertion and resource-quantity update will be treated as one unit. If either operation fails, the complete transaction will be rolled back so the database does not remain partly updated.

3. Concurrency case: before confirming a distribution I will check that the selected resource still has sufficient quantity inside the transaction. This will help prevent the same resource from being over-allocated to two distributions at the same time.

4. Integrity constraints: foreign keys will ensure that SHELTER uses an existing DISASTER_EVENT, RESOURCE uses an existing DONOR and SHELTER, REQUEST uses an existing SHELTER, and DISTRIBUTION uses an existing RESOURCE and VOLUNTEER.

5. Final validation: check for missing references, confirm that fulfilled requests match distributed quantities, and run the basic queries, joins, subqueries, view, function, procedure and transaction cases to verify the database.

Outcome of the Project

1. A normalized relational database for disaster relief resource records up to 3NF.

2. Practical implementation of DDL, DML, constraints, basic queries, aggregates, joins, subqueries, views, functions/procedures and transactions.

3. Disaster event, shelter, donation, request and distribution information can be stored and retrieved through related tables.

4. The project will give hands-on understanding of data modelling, SQL querying, integrity and transaction handling using one complete database case study.